

Software Development Proposal

InitialProposalDev

Notebook Build Audit & Execution System

Prepared for: Research & Data Science Teams
Prepared by: [Author Name / Consulting Team]
Date: July 7, 2026

Confidential — Commercial in Confidence

1 Executive Summary & Problem Statement

The Challenge

Computational notebooks (Jupyter, `.ipynb`) have become the dominant medium for data science and machine learning workflows. However, they introduce systemic risks that compromise reproducibility, correctness, and auditability at every stage of the lifecycle:

- **Untrusted notebook execution** — notebooks often originate from external collaborators, open-source repositories, or AI-assisted code generation tools without any deterministic guarantee that they run safely or correctly.
- **Lack of deterministic builds** — ad hoc environment pinning, missing dependency declarations, and absent seed controls cause notebooks to produce different results across machines or sessions, undermining scientific validity.
- **No structured audit protocol** — teams lack a systematic, multi-pass methodology for reviewing notebooks before production deployment, publication, or merge. Existing code review practices are insufficient for the unique failure modes of notebook-based ML pipelines (data leakage through preprocessing ordering, cross-validation contamination, inseparable training/inference logic).

The **Notebook Build Audit & Execution System** addresses these gaps by implementing the Construction Framework and Audit Framework defined in the **NotebookBuildAudit** specification — a formal six-pass review protocol and a three-phase construction discipline — within a containerized, AI-augmented platform.

The Objective

Build a web-based platform that enables research and data science teams to:

1. **Execute notebooks deterministically** inside Docker-sandboxed environments with pinned dependencies and reproducibility controls.
2. **Audit notebooks systematically** using the six-pass review protocol (Structural Overview, Reproducibility Check, Data Integrity Review, ML Correctness Audit, Code Quality Review, Deployment Readiness) powered by an LLM orchestration layer supporting both local (Ollama/llama.cpp) and cloud (OpenAI) providers. Additional cloud providers (Anthropic, OpenCode) are deferred to a subsequent release.
3. **Interact with notebooks programmatically** via `@jupyter-kit` for cell-level inspection, execution, and modification.
4. **Receive real-time audit logs and risk scores** (Low / Moderate / High per pass) through a modern frontend dashboard, closing the feedback loop between the Construction and Audit frameworks.

2 Project Scope & Requirements

Based on the theoretical foundations in `NotebookBuildAudit.md` and `NotebookBuildAudit.tex`, the software must include the following core features:

Notebook Construction Workbench

- Section scaffold generator that pre-populates notebooks with the canonical eight-section header structure (Environment & Dependencies, Configuration & Global Parameters, Data Ingestion, Preprocessing & Feature Engineering, Model Definition & Training, Evaluation & Metrics, Artifact Export, Conclusions & Next Steps).
- Inline reproducibility controls: random seed configuration, deterministic flag toggles, and device configuration panels.
- Versioned artifact export routing: all persisted outputs (models, plots, reports) are routed through a single, versioned export convention.

Six-Pass Audit Engine

Pass 1 — Structural Overview Produces a section map, red flag list, and recorded focus-area scope.

Pass 2 — Reproducibility Check Generates a risk score (Low / Moderate / High) based on dependency pinning, seed configuration, hardcoded paths, and end-to-end re-executability.

Pass 3 — Data Integrity Review Delivers a pipeline integrity report covering split ordering, pipeline-level leakage, missing-data handling, and ingestion-time validation.

Pass 4 — ML Correctness Audit Produces a correctness checklist for evaluation metrics, cross-validation integrity, hyperparameter tuning boundaries, and baseline comparison.

Pass 5 — Code Quality Review Outputs a code smell report flagging repetition (≥ 3 -block threshold), dead code, naming quality, and output hygiene.

Pass 6 — Deployment Readiness Returns a risk score (Low / Moderate / High) evaluating artifact export, inference/training separability, resource documentation, environment completeness, and data privacy.

LLM Orchestration Layer (Three-Level Audit)

Level 1 — Conceptual (Passes 1–2) Assesses whether the notebook is structurally coherent and re-runnable.

Level 2 — Methodological (Passes 3–4) Evaluates scientific soundness of the data pipeline and ML methodology.

Level 3 — Implementation (Passes 5–6) Assesses code maintainability and production suitability.

Hybrid provider strategy: local LLMs via Ollama API or `llama.cpp` for offline/privacy-sensitive evaluations; cloud providers (OpenAI) for high-capability inference, routed through a provider-agnostic strategy pattern that selects the appropriate model based on audit pass, latency requirements, and privacy constraints. Additional cloud providers (Anthropic, OpenCode) are supported by the extensible strategy pattern but deferred to a subsequent release.

@jupyter-kit Integration

- Programmatic notebook interaction: kernel management, cell-level execution, output capture, and metadata inspection.
- Sandbox execution pipeline: each notebook runs inside an isolated Docker container with pinned dependencies, controlled by `@jupyter-kit` cell execution commands.

Real-Time Dashboard

- Audit session management: submit notebooks, select pass scope (full six-pass or user-specified subsets), and configure LLM provider routing.
- Live audit progress: streaming per-pass deliverables as the LLM processes each level.
- Risk score visualization: Low / Moderate / High indicators per pass with drill-down to specific flagged cells.

Out of Scope

- Integration with proprietary CI/CD platforms beyond Docker-based deployment.
- Support for non-Python kernel languages (R, Julia) in the initial release.
- Automated code refactoring or rewrite suggestions — the system is diagnostic, not prescriptive.
- Multi-tenant SaaS hosting; initial deployment targets single-instance, per-team Docker Compose environments.
- Custom model fine-tuning for the LLM audit layer.

3 Proposed Technology Stack

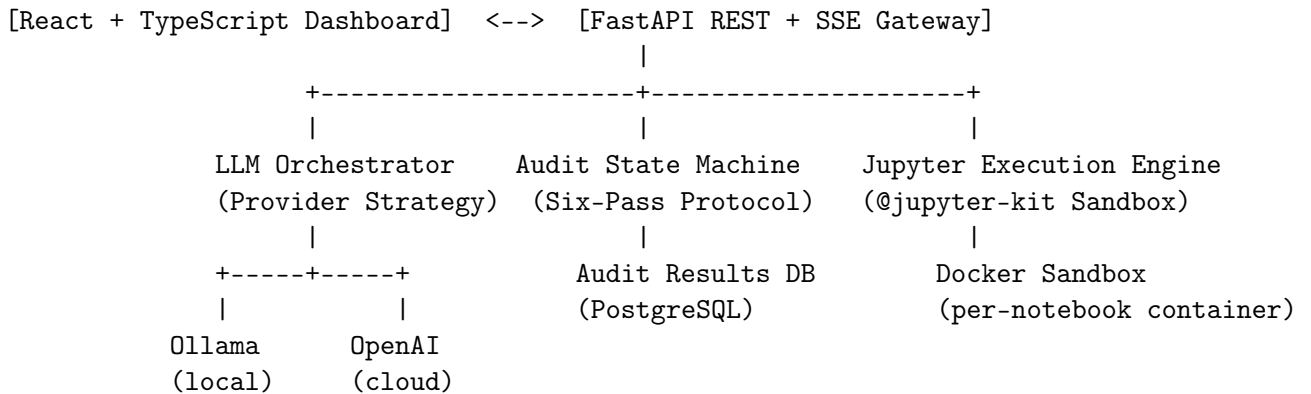
To ensure scalability, security, and high performance, I propose the following technology stack for this project:

Table 1: Technology Stack Summary

Layer	Technology & Rationale
Frontend	Vite + React 18 + TypeScript. Fast HMR (Hot Module Replacement), type-safe component development, optimal bundle splitting for real-time dashboard updates. State management via React Context or Zustand for audit session state. Styling via Tailwind CSS or CSS Modules.
Backend	Python 3.11+ + FastAPI. Async-first REST API for handling streaming LLM audit responses (Server-Sent Events), fast request validation via Pydantic models, auto-generated OpenAPI documentation.
AI / LLM Orchestration	Ollama API, <code>llama.cpp</code> , OpenAI SDK. Provider-agnostic strategy pattern with unified <code>LLMProvider</code> interface (extensible for additional providers post-launch). Supports local (Ollama/ <code>llama.cpp</code>) and cloud (OpenAI) routing. Prompt templates from <code>NotebookBuildAudit.tex</code> (Levels 1–3) are parameterized and injected per audit pass.
Notebook Execution	<code>@jupyter-kit</code> . Programmatic Jupyter kernel management, cell execution, output capture, and notebook metadata manipulation within the sandboxed Docker environment.
Database	PostgreSQL 15 + SQLAlchemy 2.0 (async). Relational storage for audit sessions, notebook metadata, audit results, and user configuration. Migration management via Alembic.
Infrastructure	Docker + Docker Compose. Fully containerized, deterministic execution environments. Each notebook audit runs in an isolated Docker container with a pinned Python environment. Docker Compose orchestrates the full stack (FastAPI, PostgreSQL, React frontend, and per-audit sandbox containers).

4 High-Level Architecture

Conceptual Diagram



A detailed technical architecture diagram, API route specifications, and data flow diagrams will be

provided upon project commencement as part of the initial deliverables.

Key Architectural Decisions

Strategy Pattern for LLM Providers A `LLMProvider` abstract base class defines `generate(prompt, **kwargs)`. Concrete implementations (`OllamaProvider`, `OpenAIProvider`) handle provider-specific API formatting. The `LLMOrchestrator` service selects the appropriate provider at runtime based on the audit configuration. The strategy pattern is designed to accept additional providers (Anthropic, OpenCode) post-launch without architectural changes.

Audit State Machine Each notebook audit is modeled as a state machine transitioning through `Scope -> Pass1 -> Pass2 -> ... -> Pass6`. Gate decisions between Levels 1, 2, and 3 are rendered as configurable checkpoints — the system can halt at a blocking issue or continue to surface further findings.

Docker Sandbox per Notebook For each audit session, the system spawns a short-lived Docker container with the notebook’s declared dependencies pre-installed. The container runs a headless Jupyter kernel managed by `@jupyter-kit`. The container is destroyed after the audit completes, providing full isolation.

Server-Sent Events (SSE) for Real-Time Audit Logs The FastAPI backend streams per-pass audit deliverables to the React frontend via SSE, enabling live progress visualization without polling.

5 Project Deliverables

Upon completion, the client will receive:

- Fully functional web application (Docker Compose deployable).
- Complete source code repository (transferred via GitHub).
- Database schema and Alembic migration scripts.
- REST API documentation (auto-generated OpenAPI/Swagger).
- Basic user documentation covering audit session creation, LLM provider configuration, and pass scope selection.
- Docker setup guide with per-notebook sandbox configuration.

6 Timeline & Milestones

The estimated time to complete this project is **8 weeks (2 months)**, broken down into the following phases:

Table 2: Project Phases

Phase	Description	Duration
Phase 1	Environment & Dockerization. Initialize monorepo structure (backend, frontend, shared schemas). Create Docker Compose configuration with FastAPI, PostgreSQL, and frontend services. Implement per-notebook Docker sandbox spawning with <code>@jupyter-kit</code> kernel management. Set up dependency pinning and environment export capture.	2 weeks
Phase 2	FastAPI Backend & LLM Integration. Implement FastAPI REST API with Pydantic models for audit sessions, notebook upload, and pass configuration. Build <code>LLMProvider</code> strategy pattern with Ollama and OpenAI concrete implementations (extensible interface ready for additional providers). Develop the <code>AuditStateMachine</code> implementing the six-pass protocol with gate decisions between Levels 1–3. Integrate <code>@jupyter-kit</code> for notebook parsing, cell execution, and output capture. Implement SSE streaming for real-time audit progress.	3 weeks
Phase 3	Frontend Interface. Scaffold Vite + React + TypeScript project. Build audit session dashboard (notebook upload, provider routing configuration, pass scope selection). Implement live audit progress view with per-pass risk score visualization and flagged-cell drill-down. Develop LLM provider configuration panel (local Ollama endpoint, cloud OpenAI API key).	2 weeks
Phase 4	Integration & QA. End-to-end integration testing with real notebooks across all six passes. Validate LLM prompt templates against reference outputs from <code>NotebookBuildAudit.tex</code> . Security review of Docker sandbox isolation and API authentication. Final documentation and deployment guide.	1 week

7 Pricing & Engagement Model

Engagement Terms

- **Team:** 2 dedicated developers allocated exclusively to this project.
- **Developer Rate:** \$1,000 USD per developer per month ($\approx$ 3,900,000 COP).
- **Communication:** Weekly syncs every Monday via Google Meet.

Investment

Table 3: Total Cost Breakdown

Cost Item	USD	COP (approx.)
Developer Costs (2 devs × 2 months × \$1,000/mo)	\$4,000	≈ 15,600,000 COP
Hardware / Server PC	\$1,000	≈ 3,900,000 COP
Total Project Cost	\$5,000	≈ 19,500,000 COP

Payment Schedule

- 1. **30% Upfront deposit** to start development (≈ \$1,500 USD / ≈ 5,850,000 COP).
- 2. **40% Upon completion of Phase 2** — Backend & LLM Integration (≈ \$2,000 USD / ≈ 7,800,000 COP).
- 3. **30% Upon final delivery and deployment** (≈ \$1,500 USD / ≈ 5,850,000 COP).

8 Maintenance & Support

Following the final deployment, this proposal includes **2 months** of complementary technical support.

Included: Bug fixes, security patches, and minor UI adjustments.

Not included: Development of new features (these can be quoted separately or handled via a monthly retainer after the complementary period ends).

9 Next Steps & Acceptance

To proceed with this proposal, please sign below or confirm via email. Upon acceptance, I will send over the formal contract and the initial invoice.

Accepted by: Date: